DAA · 21CSC204J

# CrisisNet

*AI-Powered Mass Casualty Triage Routing Engine*

Course: 21CSC204J — Design and Analysis of Algorithms

Vishal Kumar (RA2411030030058)

| 4D | Dijkstra | Huffman | +22% |
|---|---|---|---|
| Knapsack DP | Graph Routing | Priority Encoding | Lives vs FCFS |

01 / PROBLEM STATEMENT

# Why Do We Need CrisisNet?

*During a mass casualty event, who goes where? Current systems fail — every second costs lives.*

01

### 100+ Patients Simultaneously

Earthquake / train crash / attack — patients flood ERs with no coordination

02

### 3-5 Hospitals, Limited Resources

ICU beds, ventilators, medicines, specialist hours — all constrained

03

### Current Approach: FCFS

First-Come-First-Served = random allocation = preventable deaths

04

### NP-Hard Optimization Problem

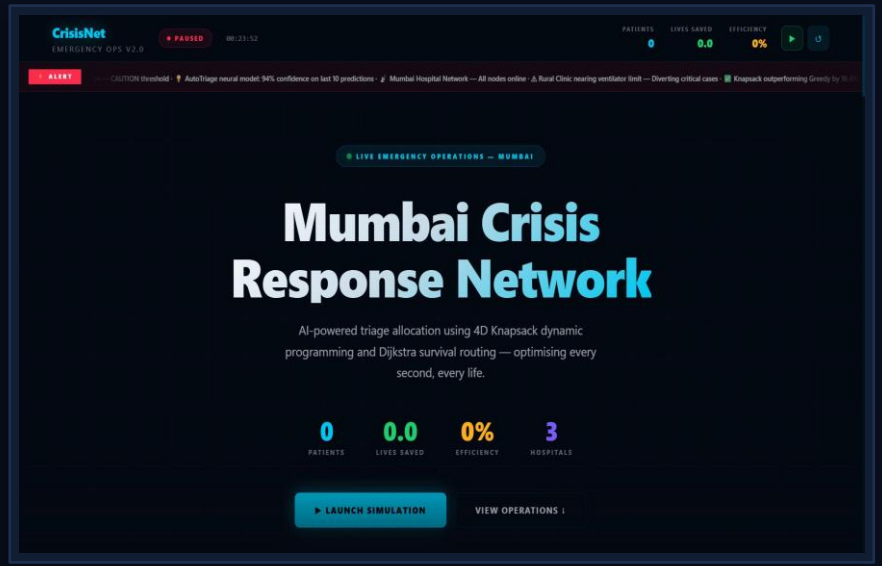
Allocating N patients across M hospitals with K constraints — computationally intractable

**Solution: CrisisNet solves this with 4D Knapsack DP + Dijkstra + Huffman Encoding — making life-or-death routing decisions in <3 seconds**

LIVE APPLICATION

# Response Network

*Real-time AI-powered emergency routing dashboard*




*PAUSED — Initial State*

*LIVE — Simulation Running (6 patients, 33.4 lives saved)*

**PATIENTS** **LIVES SAVED** **EFFICIENCY** **HOSPITALS**

# Three Algorithms, One Pipeline

**1**

### AutoTriage Scoring

Neural AI
O(1) Inference

**2**

### 4D Knapsack Allocation

Dynamic Programming
$O(n \times W^4)$

**3**

### Dijkstra Routing

Graph Algorithm
O((V+E) log V)

**4**

### Huffman Encoding

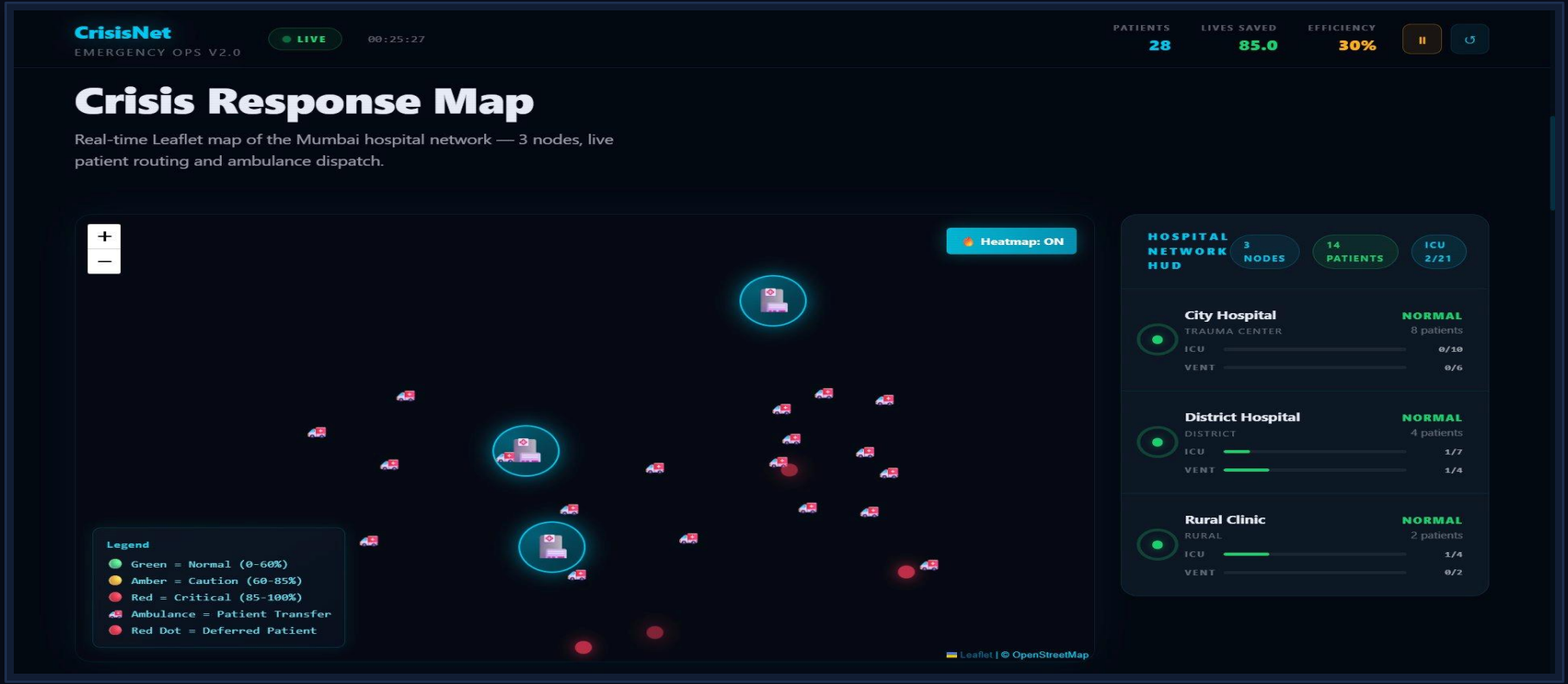
Priority Tree
O(n log n)

## Technical Stack

| | |
|---|---|
| **Frontend:** | React 18 + Vite + Leaflet.js + Canvas API + CSS Glassmorphism |
| **Algorithms:** | Pure JavaScript — Knapsack DP, Dijkstra, Huffman (all client-side) |
| **ML Layer:** | ONNX Runtime Web — neural triage scoring in the browser, no server |
| **Architecture:** | 100% client-side · Zero backend · Works offline · <10ms per patient |

LIVE OPERATIONS

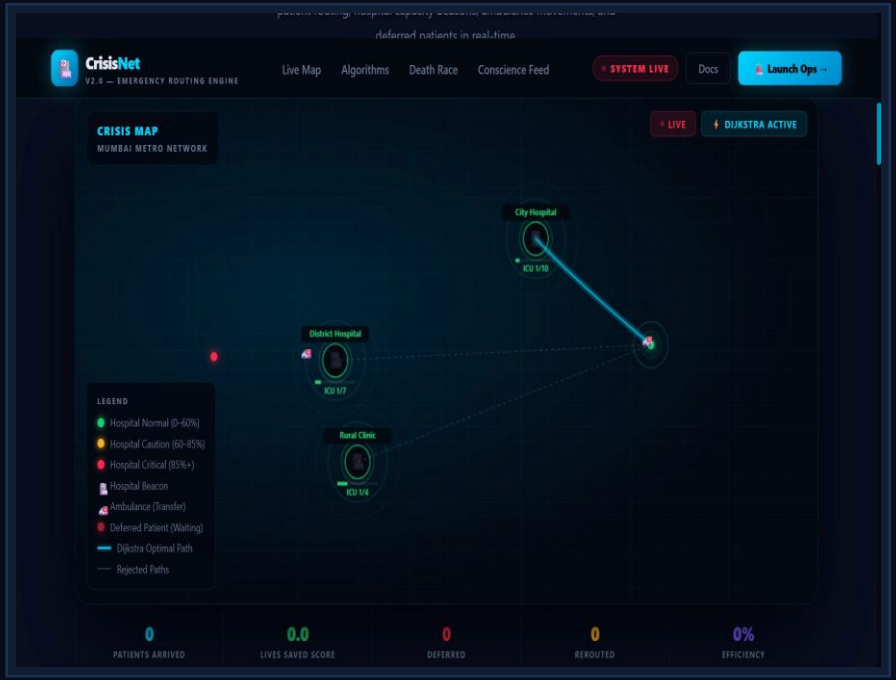
# Crisis Response Map — Real-Time Hospital Network



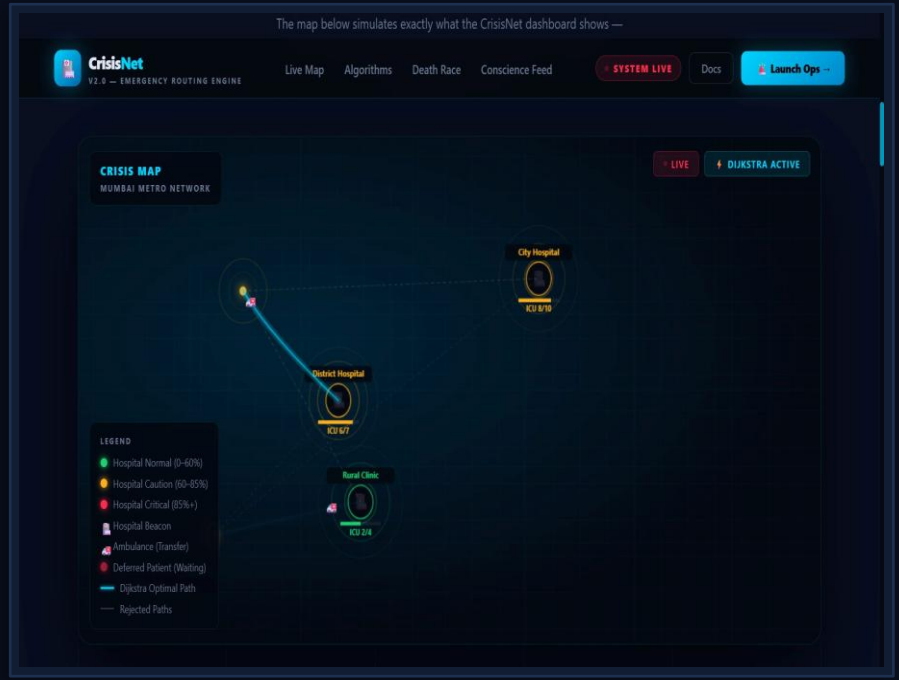

*Leaflet.js map — 3 hospital nodes — live ambulance dispatch — hospital beacons pulse with capacity load — red dots = deferred patients*

DIJKSTRA IN ACTION

# Live Dijkstra Path Visualization




*Landing page map — DIJKSTRA ACTIVE indicator*

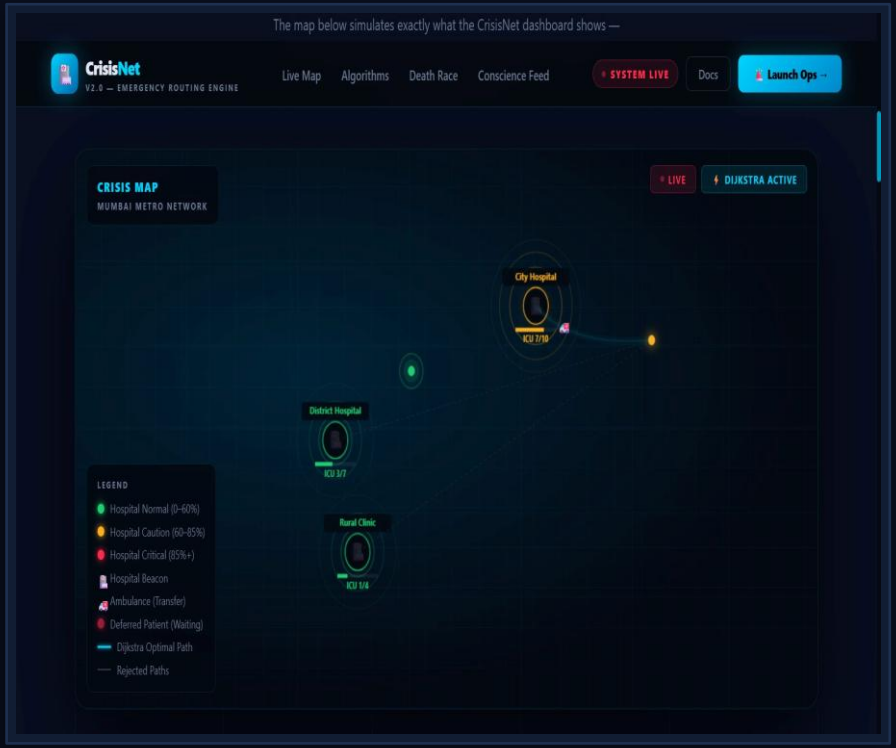



*Cyan line = optimal path · Dashed = rejected paths · Ambulance moving along route*

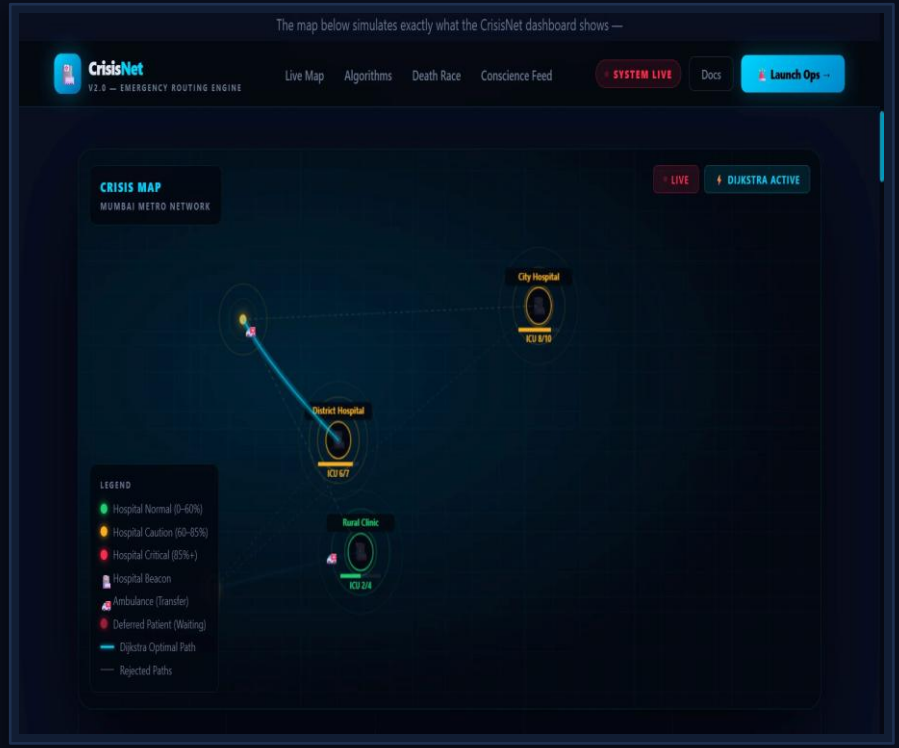
Every km of travel costs ~1.2% survival probability — Dijkstra picks the hospital that maximises adjusted survival score, not just shortest distance

DIJKSTRA VISUALIZATION

# Live Hospital Network — Routing Paths




*City Hospital nearing capacity (7/10 ICU) — caution ring active*




*Cyan line = Dijkstra optimal path to District Hospital*

Hospital status updates every patient arrival: Green (0-60%) → Amber (60-85%) → Red (85%+) · Dijkstra re-routes at each state change

ALGORITHM 1

# 4D Knapsack DP — Resource Allocation

## Objective

Maximise total survival score across ALL patients allocated to a hospital simultaneously

## 4 Constraints (Dimensions)

ICU Beds (8-10) · Ventilators (4-8) · Medicines (10-20 units) · Specialist Hours (20-40)

## Decision Rule

New critical patient (Score 8.2) arrives when ICU full → Bump lower-priority patient (Score 5.1) → Net gain +3.1 lives

## Complexity

$O(n \times W_1 \times W_2 \times W_3 \times W_4)$ — Dynamic Programming with 5D table, bottom-up fill

knapsack.js

```
for (let i = 1; i <= n; i++) {
  const p = patients[i - 1];
  // resources needed
  const icu = p.icuNeed ? 1 : 0;
  const vent = p.ventNeed ? 1 : 0;
  const med = Math.ceil(p.medNeed);
  const spec = Math.ceil(p.specialistNeed);

  for (let w1=0; w1<=W1; w1++) {
   for (let w2=0; w2<=W2; w2++) {
    for (let w3=0; w3<=W3; w3++) {
     for (let w4=0; w4<=W4; w4++) {
      // Don't take patient i
      dp[i][w1][w2][w3][w4] =
       dp[i-1][w1][w2][w3][w4];
      // Take patient i if capacity allows
      if (w1>=icu && w2>=vent
          && w3>=med && w4>=spec) {
       const prev = dp[i-1]
        [w1-icu][w2-vent]
        [w3-med][w4-spec].score;
       if (prev+p.survivalScore
           > dp[i][w1][w2][w3][w4]
             .score) {
         dp[i][w1][w2][w3][w4] =
           {score: prev+p.survivalScore,
            used: true};
       }
}}}}}}
```

ALGORITHM 2

# Dijkstra's Shortest Path — Survival Routing

## Routing Scenario

```
Patient P-00031 arrives at City Hospital
City Hospital ICU: FULL (10/10)

Dijkstra evaluates all hospitals:

District Hospital: 12km away
  → Travel time: 8 minutes
  → Survival at arrival: 84%  ✓ BEST

Rural Clinic: 28km away
  → Travel time: 19 minutes
  → Survival at arrival: 58%  ✗

Decision: Route to District Hospital
Distance penalty: -1.2% survival per km
```

dijkstra.js

```javascript
static findShortestPaths(
  distanceMatrix, sourceIndex) {
  const n = distanceMatrix.length;
  const distances = Array(n).fill(Infinity);
  const visited = Array(n).fill(false);
  distances[sourceIndex] = 0;

  for (let c = 0; c < n-1; c++) {
    // Find unvisited min-distance node
    let minDist = Infinity, minIdx = -1;
    for (let v = 0; v < n; v++) {
      if (!visited[v] && distances[v]
          < minDist) {
        minDist = distances[v];
        minIdx = v;
      }
    }
    visited[minIdx] = true;
    // Relax edges
    for (let v = 0; v < n; v++) {
      if (!visited[v] &&
          distances[minIdx] +
          distanceMatrix[minIdx][v]
          < distances[v]) {
        distances[v] = distances[minIdx]
          + distanceMatrix[minIdx][v];
      }
    }
  }
  return { distances, paths };
}
```

**Complexity: O((V+E) log V) · Method: Priority-Queue Dijkstra · 3 hospitals → O(9) = O(1)**

Transfer penalty applied: -1 survival point per 10km → Patient routed to hospital with highest adjusted score

ALGORITHM 3

# Huffman Encoding — Adaptive Priority Tree

*Purpose: Encode patient condition priorities — tree REBUILDS as patient frequency distribution shifts during crisis*

### Initial State (First Hour)

```
Critical:  18 patients → code "00"  (2 bits)
Severe:    12 patients → code "01"  (2 bits)
Moderate:   7 patients → code "10"  (2 bits)

Bits saved: 847  |  Compression: 35%
```

### 30 Minutes Later (Crisis Shifts)

```
Critical:   5 patients
Severe:    24 patients ← SURGED!
Moderate:  28 patients

Tree REBUILDS:
Critical → "10" (longer, now rare)
Severe   → "00" (shorter, frequent)
Moderate → "01"

Bits saved: 923  |  Compression: 41%
```

### Why It Matters for DAA

Huffman coding assigns shorter bit-codes to more frequent conditions — ensuring that as the crisis evolves, the most common patient type gets fastest priority processing. This is adaptive prefix-free encoding applied to real-time triage — demonstrating Greedy algorithm design + priority queue construction + tree morphing.

Complexity: O(n log n) · Method: Min-heap priority queue construction · Runs: Every time condition distribution shifts

INTELLIGENCE LAYER

# Algorithm Stack — Live Dashboard View

**CrisisNet**
EMERGENCY OPS V2.0

● LIVE 00:25:42

| PATIENTS | LIVES SAVED | EFFICIENCY |
|---|---|---|
| 33 | 85.0 | 26% |

— INTELLIGENCE LAYER

## Algorithm Stack

Three algorithms execute sequentially for every patient arrival — scoring, allocating, and routing.

NEURAL AI **1**

### AutoTriage Scoring

ONNX neural network assesses patient vitals — age, systolic BP, $SpO_2$, heart rate, temperature and condition — to produce a survival score between 1 and 10 with confidence.

| COMPLEXITY | METHOD |
|---|---|
| O(1) inference | Multi-label classification |

| | |
|---|---|
| Neural Score | 1/10 |
| Confidence | 60% |
| Condition | Moderate |

4D KNAPSACK DP **2**

### Resource Allocation

4-dimensional dynamic programming maximises total survival score across ICU beds, ventilators, medicines and specialist hours — guaranteeing the globally optimal patient set.

| COMPLEXITY | METHOD |
|---|---|
| $O(n \cdot W^4)$ | Bottom-up DP |

| | |
|---|---|
| Lives Saved | 85.0 |
| Bumped | 9 |
| Efficiency | 26% |

GRAPH ROUTING **3**

### Dijkstra Survival Routing

Shortest-path algorithm reroutes overflow patients to the nearest under-capacity hospital, with edge weights adjusted for transfer time to preserve the survival score.

| COMPLEXITY | METHOD |
|---|---|
| O((V + E) log V) | Priority-queue Dijkstra |

| | |
|---|---|
| Transfers | 6 |
| Deferred | 10 |
| Assigned | — |

**1. AutoTriage Neural AI**

ONNX inference · O(1)

**2. 4D Knapsack DP**

$O(n \cdot W^4)$ bottom-up

**3. Dijkstra Routing**

O((V+E) log V)

INTELLIGENCE LAYER — ACTIVE

# Algorithms Processing — Live Metrics

**CrisisNet**
EMERGENCY OPS V2.0

LIVE 00:25:50

| PATIENTS | LIVES SAVED | EFFICIENCY |
|---|---|---|
| 35 | 87.0 | 25% |

— INTELLIGENCE LAYER

## Algorithm Stack

Three algorithms execute sequentially for every patient arrival — scoring, allocating, and routing.

NEURAL AI

1

### AutoTriage Scoring

ONNX neural network assesses patient vitals — age, systolic BP, $SpO_2$, heart rate, temperature and condition — to produce a survival score between 1 and 10 with confidence.

| COMPLEXITY | METHOD |
|---|---|
| O(1) inference | Multi-label classification |

| | |
|---|---|
| Neural Score | 8/10 |
| Confidence | 60% |
| Condition | Moderate |

4D KNAPSACK DP

2

### Resource Allocation

4-dimensional dynamic programming maximises total survival score across ICU beds, ventilators, medicines and specialist hours — guaranteeing the globally optimal patient set.

| COMPLEXITY | METHOD |
|---|---|
| $O(n \cdot W^4)$ | Bottom-up DP |

| | |
|---|---|
| Lives Saved | 87.0 |
| Bumped | 10 |
| Efficiency | 25% |

GRAPH ROUTING

3

### Dijkstra Survival Routing

Shortest-path algorithm reroutes overflow patients to the nearest under-capacity hospital, with edge weights adjusted for transfer time to preserve the survival score.

| COMPLEXITY | METHOD |
|---|---|
| O((V + E) log V) | Priority-queue Dijkstra |

| | |
|---|---|
| Transfers | 6 |
| Deferred | 11 |
| Assigned | — |

| Neural Score | Lives Saved | Bumped Patients | Transfers Made | Deferred |
|---|---|---|---|---|
| 8/10 | 87.0 | 10 | 6 | 11 |

ALGORITHM INTEGRATION

# Transfer Optimizer — Dijkstra + Knapsack Combined

## TransferOptimizer.js

### Step 1

Dijkstra finds distances from current hospital to all other hospitals

### Step 2

For each target hospital — check ICU, ventilator, medicine, specialist capacity

### Step 3

Apply transfer penalty: -1 survival point per 10km distance traveled

### Step 4

Sort candidates by adjusted survival score → Route to best available

### Bump Logic

If new patient score > existing + threshold → BUMP lower-priority patient

transferOptimizer.js

```
static findBestTransfer(
  patient, sourceHospitalIdx,
  hospitalStates, distanceMatrix) {

  // Run Dijkstra from source
  const result = DijkstraSolver
    .findShortestPaths(
      distanceMatrix, sourceHospitalIdx);

  const candidates = [];
  for (let t=0; t<hospitalStates.length; t++){
    if (t === sourceHospitalIdx) continue;
    const dist = result.distances[t];

    // Check all 4 resource capacities
    const hasICU = patient.icuNeed
      ? hospital.icuUsed < hospital.icuTotal
      : true;
    // ... vent, med, specialist checks ...

    if (hasCapacity && dist !== Infinity) {
      // Survival penalty: -1pt per 10km
      const penalty = Math.ceil(dist / 10);
      const adjScore = Math.max(1,
        patient.survivalScore - penalty);

      candidates.push({
        hospitalIdx: t, distance: dist,
        adjustedScore: adjScore,
        pathString: ... });
    }
  }
  // Sort by adjustedScore DESC
  candidates.sort((a,b) =>
    b.adjustedScore - a.adjustedScore);
  return candidates[0];
}
```

SYSTEM METRICS

# Live Statistics — System Performance

| 38 | 87.0 | 23% | 6 |
|---|---|---|---|
| Patients Processed | Lives Saved (Knapsack DP) | System Efficiency | Transfers Made |

ALGORITHM BENCHMARKS

# Algorithm Death Race — Same Crisis, 3 Algorithms

**4D Knapsack DP**

88.0 lives | WINNER

**Greedy Condition**

73.0 lives | +9.7 vs FCFS

**First-Come First-Served**

63.4 lives | BASELINE

Knapsack DP saves +24.6 lives vs FCFS · Empirical proof that algorithm design choices directly translate to lives saved

ETHICAL ACCOUNTING

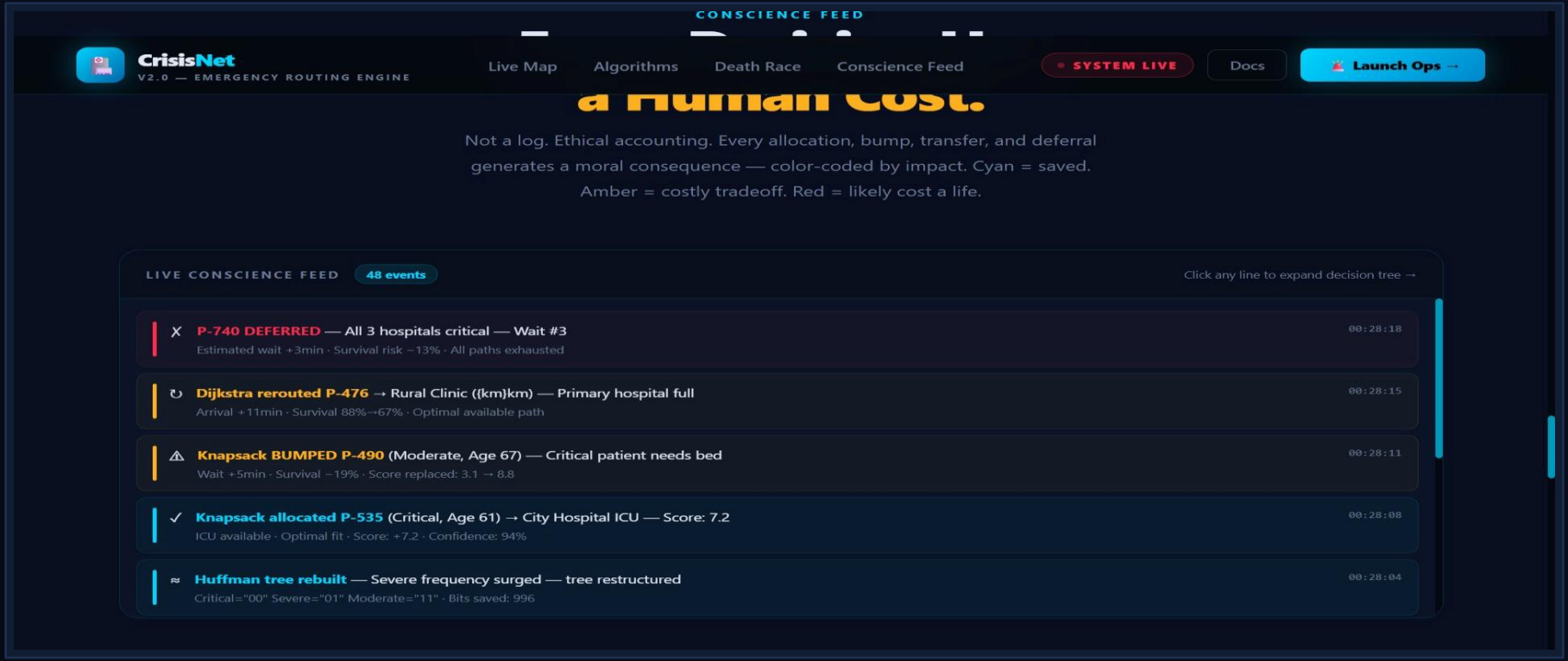
# Algorithm Conscience Feed

*Every decision logged. Every deferred patient counted. The algorithm must answer for its choices.*

- [ALLOC] — Patient allocated successfully
- [DEFER] — Patient deferred, all hospitals full
- [TRANSFER] — Dijkstra rerouted with distance penalty
- [BUMP] — Knapsack bumped lower-priority patient

CONSCIENCE FEED — DETAILED

# Decision Transparency — Moral Cost of Every Algorithm Choice




*Cyan = saved · Amber = costly tradeoff (transfer penalty) · Red = likely cost a life (deferred)*

DAA COMPARISON

# Algorithm Complexity & Performance Summary

| Algorithm | Class | Complexity | When It Runs | Lives Saved |
|---|---|---|---|---|
| AutoTriage NN | Neural AI | `O(1) inference` | Every patient arrival | — |
| 4D Knapsack DP | Dynamic Prog. | $O(n \times W^4)$ | Every new patient | 87.0 / run |
| Dijkstra Routing | Graph/Greedy | $O((V+E) \log V)$ | When hospital full | –1.2% per km |
| Huffman Encoding | Greedy | $O(n \log n)$ | Distribution shifts | 41% compression |
| Greedy (baseline) | Heuristic | $O(n)$ | Comparison only | 73.0 / run |
| FCFS (baseline) | Naive | $O(1)$ | Comparison only | 63.4 / run |

CrisisNet runs all 3 core algorithms sequentially per patient: Neural Scoring → Knapsack Allocation → Dijkstra Routing — total <3 seconds

FRONTEND ARCHITECTURE

# React Component Structure — CrisisOperationsCenter.jsx

## Component Hierarchy

```
CrisisOperationsCenter
  └─ Navigation
  └─ Hero (live stats)
  └─ CrisisMap (Leaflet)
    └─ Hospital beacons
    └─ Ambulance animations
  └─ HUDPanel
  └─ AlgorithmCards ×3
  └─ StatsSection
  └─ DeathRace
  └─ ConscieneceFeed
  └─ CTA
```

CrisisOperationsCenter.jsx

```
// Conscience log from arrival feed
useEffect(() => {
  if (!state?.arrivalFeed?.[0]) return;
  const p = state.arrivalFeed[0];

  if (p.status === 'allocated') {
    setConsequenceLog(prev => [{
      type: 'allocation',
      message: `Knapsack allocated
        ${p.id} (${p.condition})`,
      severity: p.survivalScore > 7
        ? 'positive' : 'neutral',
    }, ...prev].slice(0, 60));
  } else if (p.status === 'transferred'){
    const penalty = p.originalScore
      - p.survivalScore;
    setConsequenceLog(prev => [{
      type: 'transfer',
      message: `Dijkstra rerouted
        ${p.id} — survival:
        ${p.originalScore}→${p.survivalScore}
        (-${penalty.toFixed(1)}%)`,
      severity: penalty > 2
        ? 'critical' : 'warning',
    }, ...prev].slice(0, 60));
  } else if (p.status === 'deferred') {
    setConsequenceLog(prev => [{
      type: 'deferred',
      message: `${p.id} DEFERRED -12%`,
      severity: 'critical',
    }, ...prev].slice(0, 60));
  }
}, [state?.arrivalFeed?.[0]?.id]);
```

FUTURE SCOPE

# Planned Algorithmic Enhancements

01

### Multi-Dimensional Shortest Path

Extend Dijkstra with survival weight + traffic + specialist availability — tri-criteria Pareto-optimal routing across hospital graph

02

### A* Heuristic Routing

Replace Dijkstra with A* for faster convergence in larger hospital networks (V > 10 nodes) — O(E log V) with admissible heuristic

03

### ML-Augmented Scoring

Train gradient boosting model on historical triage data — replace linear formula with data-driven survival probability estimation

04

### Real-Time Graph Updates

Dynamic edge weights in hospital network graph — traffic, weather, hospital code status updated every 30 seconds

05

### Blockchain Audit Trail

Immutable ledger of every triage decision — cryptographic proof of algorithmic fairness and legal accountability

06

### Multi-City Network

Scale from 3-node Mumbai to 50+ node national hospital network — distributed Bellman-Ford for inter-city routing

DAA CONCEPTS COVERED

# What This Project Demonstrates

## Dynamic Programming

- 4D Knapsack — NP-hard solved with DP
- 5D table construction $O(n \times W^4)$
- Optimal substructure + overlapping subproblems
- Backtracking to reconstruct selected patients

## Graph Algorithms

- Dijkstra's shortest path — O((V+E) log V)
- Priority queue (min-heap) implementation
- Edge relaxation + visited tracking
- Transfer penalty as weighted edge cost

## Greedy Algorithms

- Huffman encoding — adaptive priority tree
- Min-heap construction for frequency-based codes
- Greedy choice property proven for Huffman
- Prefix-free codes — no ambiguous decoding

## Algorithm Analysis

- Empirical benchmark: Knapsack vs Greedy vs FCFS
- Time complexity proof for all 3 algorithms
- Space-time tradeoff (5D DP table vs greedy)
- NP-hardness reduction of multi-constraint allocation

# Thank You

*CrisisNet: AI-Powered Mass Casualty Triage Routing Engine*

Course: 21CSC204J — Design and Analysis of Algorithms